# CPU vs GPU Hardware Utilization Report

GCMC, FunkSVD and LLORMA on MovieLens - 100K

**Objective :**

This report analyzes hardware-utilization data collected while training three recommendation algorithms — GCMC (Graph Convolutional Matrix Completion), FunkSVD, and LLORMA (Local Low-Rank Matrix Approximation) — on both CPU and GPU, using the MovieLens-100K dataset. The goal is to explain what each collected metric means, present the measured values, and interpret why GPU acceleration helped some algorithms and hurt others.

**Setup :**

For all the Three algorithms i implemented a custom HardwareMonitor class, in which i implemented in such a way that it runs concurrently with training and does not affect training of the models and it for the whole duration of the training it samples the metrics every 0.2 seconds and after each time the training completes it calculates the average of the metrics and it repeats the same process five times and takes the average of all the values.
I performed five independent runs because a single execution can be affected by other overheads such as GPU kernel launches, memory operations, scheduling, and other system activity. By repeating the experiment and taking the mean, we obtain a more representative measurement and can also report the standard deviation to show the variation between runs."

I also considered standard deviation apart from average/mean because it tells us how consistent our results are from each other.

| ALGORITHM | FRAMEWORK | TRAINING APPROACH | EPOCHS / RUNS | CODE && CSV REFERENCE |
|---|---|---|---|---|
| FunkSVD | PyTorch (nn.Embedding) | Full-batch gradient descent on user/movie latent factors + biases | 30 epochs × 5 runs | GCMC_… |
| GCMC | PyTorch | Full-batch training on a bipartite user-movie graph; one linear layer per rating class, scatter-add (index_add_) message aggregation | 20 epochs × 5 runs | GCMC_F… |

| | | | | |
|---|---|---|---|---|
| LLORMA | NumPy (CPU) / CuPy (GPU) | 20 independent local low-rank models trained sequentially, each around a randomly chosen anchor point, then kernel-weighted and blended at inference | 15 epochs/anchor, 20 anchors × 5 runs | LLORMA… |

**METRICS I USED :**

1. **CPU Utilization**
   - Measures how much of the available CPU capacity is being used by the training process at the given snapshot.
   - It helps us understand how heavily the algorithm depends on CPU computation.
2. **GPU Utilization**
   - Measures how actively the GPU's compute resources are being used during training.
   - It helps us understand whether the GPU is being effectively utilized by the algorithm.
3. **CPU RAM Usage**
   - Measures the amount of system memory occupied by the Python training process.
   - It helps us compare the memory requirements of different algorithms when running on the CPU.
4. **GPU VRAM Usage**
   - Measures how much GPU memory is occupied by the model, data, and other GPU-related objects during training.
   - It helps us understand the GPU memory requirement of each algorithm.
5. **Training Time**
   - Measures the total wall-clock time required to complete the training process.
   - This is the main performance metric for comparing how fast the different implementations are.

**I also Calculated :**

6. **GPU Power Consumption**
   - Measures the instantaneous power drawn by the GPU during training, in Watts.
   - This helps us understand the power requirement of the GPU implementation.

7. **GPU Energy Consumption**
    - Estimates the total energy consumed by the GPU during the training run, in Joules.
    - It is calculated using the GPU power measurements collected over time.
8. **GPU Temperature**
    - Records the GPU temperature during training.
    - This provides an additional indication of the thermal behavior of the GPU.

**HARDWARE UTILIZATION RESULTS:**

# FunkSVD

| Metric | CPU | GPU |
|---|---|---|
| Avg CPU util (%) | 73.5 | 17.625 |
| Avg RAM (MB) | 797 | 1276 |
| Avg GPU util (%) | — | 33.6 |
| Avg VRAM (MB) | — | 628 |
| Avg GPU power (W) | — | 27.4 |
| Avg GPU temp (°C) | — | 48.9 |
| GPU energy (J) | — | ~0 |
| Training time (s) | 1.96 | 0.11 |

FunkSVD is the clear GPU winner: FunkSVD shows the **biggest improvement on GPU**. The training time goes from **1.96 seconds on CPU to 0.11 seconds on GPU**, which is about **17 times faster**.

The main reason is that FunkSVD mainly performs simple operations such as embedding lookups and dot products for the user-movie pairs. These operations can be done very efficiently in parallel on the GPU. Since a large batch of around 100K ratings is processed together in each epoch, the GPU gets enough work to do at the same time.

So, unlike GCMC and LLORMA, FunkSVD has a workload that is well suited for GPU processing. The GPU can perform many of the calculations in parallel, which greatly reduces the training time.

# GCMC

| Metric | CPU | GPU |
|---|---|---|
| Avg CPU util (%) | 39.2 | 40.7 |
| Avg RAM (MB) | 699 | 1221 |
| Avg GPU util (%) | — | 7.1 |
| Avg VRAM (MB) | — | 609 |
| Avg GPU power (W) | — | 23.8 |
| Avg GPU temp (°C) | — | 47.8 |
| GPU energy (J) | — | 102.3 |
| Training time (s) | 3.93 | 3.76 |

GCMC shows only a marginal GPU speedup : GCMC GPU training is almost the same as CPU training, with **3.93 seconds on CPU and 3.76 seconds on GPU**, so the GPU is only about **4% faster**. The GPU utilization is also very low, at around **7.1% on average**.

The main reason is that GCMC uses scatter and index_add_ operations for message passing. The graph used here is also fairly small, with around 2,600 nodes. Because of this, the GPU does not have enough large work to keep its computing units busy. A lot of the time is spent on memory access, starting small GPU operations, and waiting for them to finish instead of doing large matrix calculations.

So, even though GCMC is running on the GPU, the GPU is not being used much. The CPU can handle these smaller operations without the extra GPU overhead, which is why the CPU and GPU training times are almost the same in this case.

# LLORMA

| Metric | CPU | GPU |
|---|---|---|
| Avg CPU util (%) | 38.2 | 41.1 |
| Avg RAM (MB) | 187 | 569 |
| Avg GPU util (%) | — | 13.9 |
| Avg VRAM (MB) | — | 488 |

| | | |
|---|---|---|
| Avg GPU power (W) | — | 23.3 |
| Avg GPU temp (°C) | — | 36.7 |
| GPU energy (J) | — | 323.5 |
| Training time (s) | 6.62 | 12.06 |

LLORMA is the standout negative result: LLORMA GPU training is around **1.8 times slower than CPU training** (6.62 s → 12.06 s). GPU usage is low, and the average GPU usage is only 13.9%

The main reason is how LLORMA is implemented. It trains 20 small local models one after another using a Python for loop. Each local model works on only a small part of the training data. Because the models are small, the GPU has very little work to do at one time.
Also, each small model needs several small GPU operations. These operations have some extra time for starting the GPU work and moving/syncing data between the CPU and GPU. The cp.add.at operation used in the code also involves many small and irregular updates, which does not make good use of the GPU.

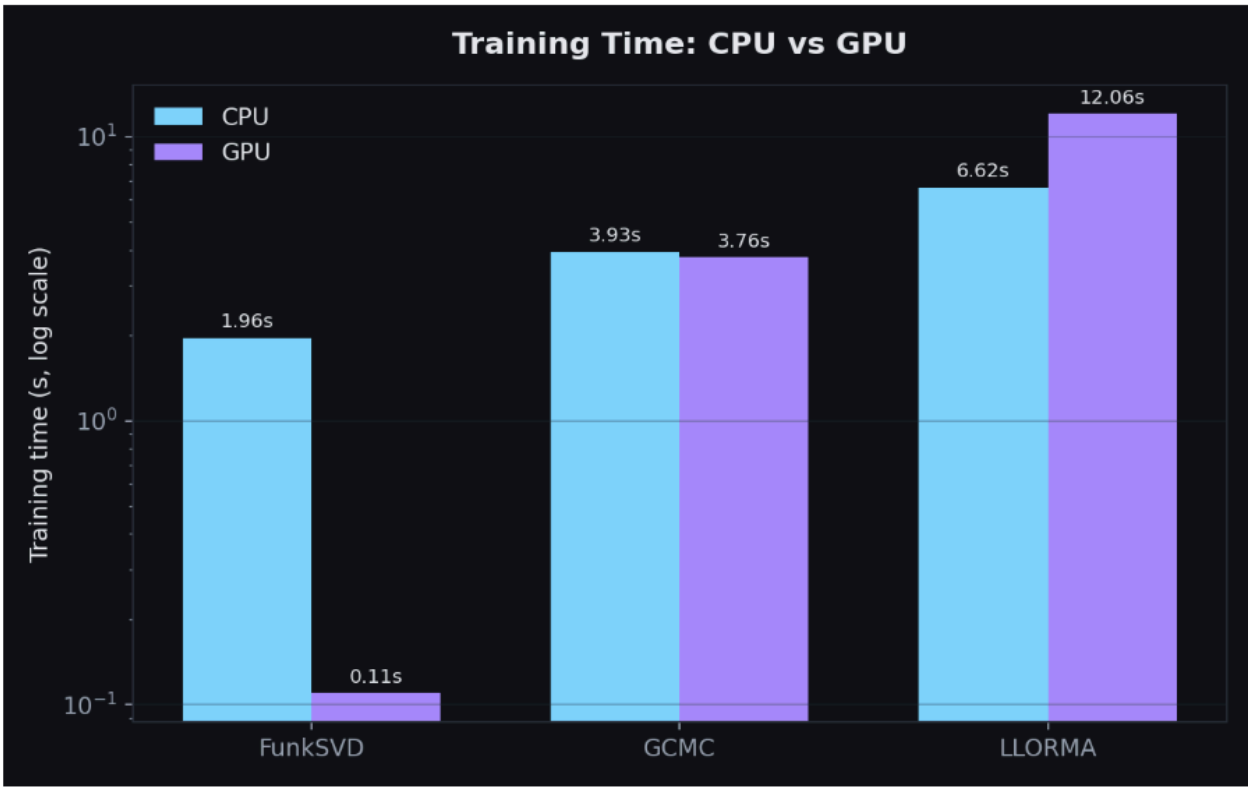
So, the GPU spends a lot of time starting and waiting for small operations instead of doing large amounts of work. The CPU does not have this extra GPU launch and synchronization time, so it finishes the same LLORMA training faster.

Simply We Can say that : LLORMA's workload is too small and split into many small operations, so the GPU cannot use its full power.

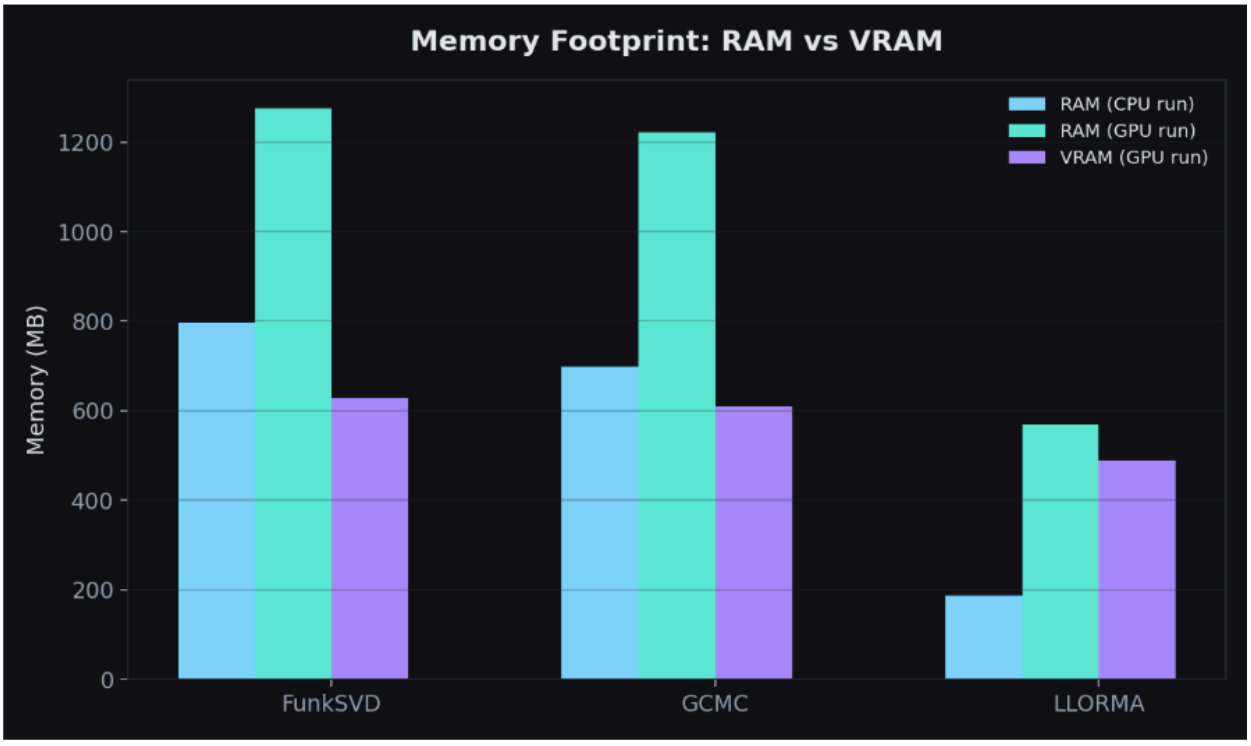
DATASET LINK : MOVIE_LENS-100K

Various Visualizations of all three Model comparisons are listed Below:

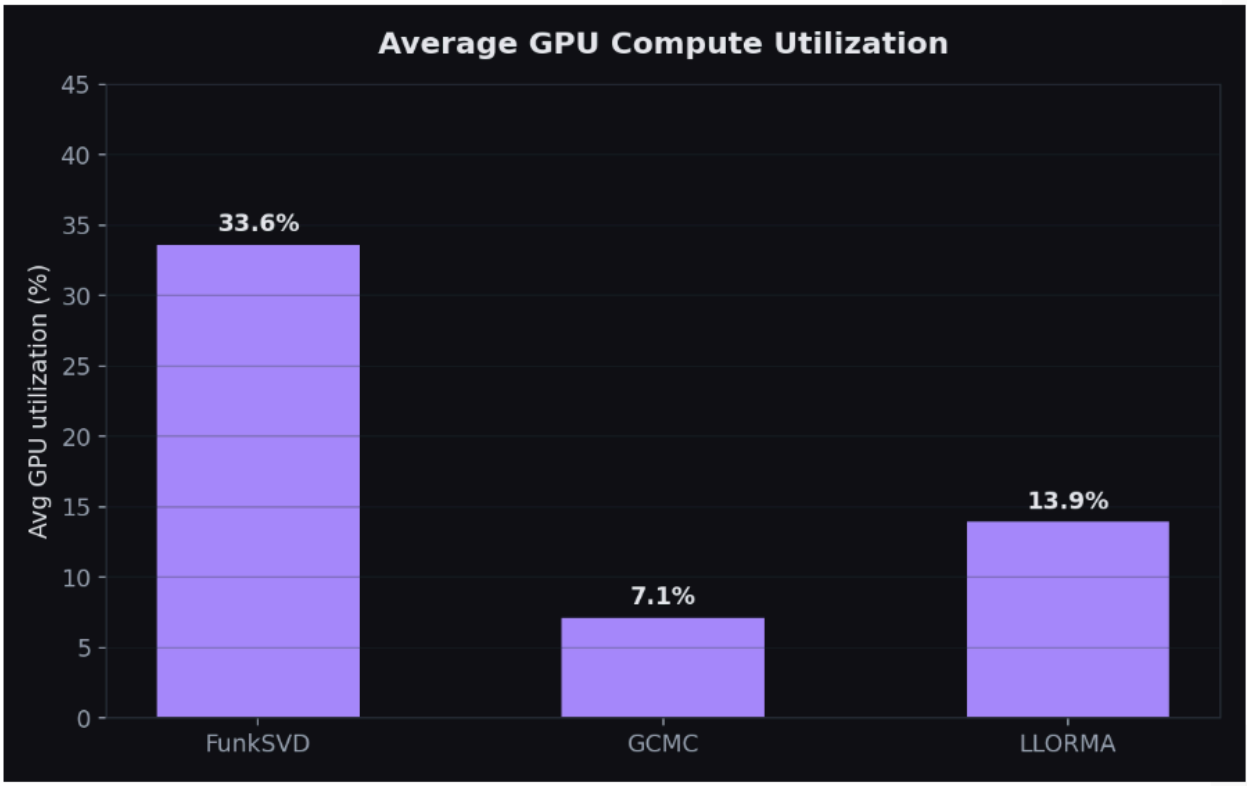
**TRAINING TIME :**




**MEMORY USAGE IN ALL THREE MODELS :**



**AVERAGE GPU COMPUTE UTILIZATION :**




## Accuracy Results

These are the model-quality numbers computed after training,

| Model | Device | RMSE | MAE | Simple Explanation |
|---|---|---|---|---|
| FunkSVD | CPU / GPU | ~ 2.8 – 2.9 | — | Plain regression from independent latent factors + biases |
| GCMC (baseline) | CPU / GPU | ~ 1.05 | — | Classification over rating buckets using graph structure |
| GCMC (Enhanced, +side features) | GPU | 1.0847 | — | Adds user rating-behavior stats + movie genre one-hot features |
| LLORMA | CPU | 1.4531 | 1.0978 | Inference time 0.098s |

| | | | | |
|---|---|---|---|---|
| LLORMA | GPU | 1.4824 | 1.1342 | Inference time 1.118s (GPU inference also slower) |

1) GCMC gives better RMSE than both FunkSVD and LLORMA. This suggests that, for this dataset, using the graph information helps GCMC learn better than the simple factorization used by FunkSVD and the local models used by LLORMA.
2) The enhanced GCMC also gives better results than the baseline GCMC. It uses extra information such as movie genres and user behavior. The enhanced version has an RMSE of 1.0847 however it reaches the final training loss of the baseline about 3 epochs earlier. This shows that the additional features can help the model learn faster.
3) LLORMA on GPU performs worse than CPU in the measured results. It takes more time for both training and inference, and its RMSE/MAE are also slightly worse. However, the small difference in RMSE/MAE may be because of the difference between runs. So, if we want to make a strong claim about the accuracy difference, we should run both versions again using the same random seeds.

## Conclusion

Across the three algorithms, GPU does not always give better performance. It mainly depends on how the work is done.

- FunkSVD gets the biggest benefit from the GPU, with around 17× faster training. This is because it performs many similar calculations on the full dataset, which the GPU can process in parallel very well.
- GCMC gets very little improvement from the GPU. Its graph message-passing uses many small scatter operations. Because the operations are small, the GPU is not fully used and the extra GPU overhead reduces the benefit.
- LLORMA becomes slower on the GPU. It trains 20 small local models one after another. Since each model performs only a small amount of work, the time needed to start and synchronize the GPU operations becomes more important than the actual GPU computation.

For accuracy, GCMC gives the best results among the three, especially the enhanced GCMC that uses additional user and movie features. So overall, GCMC looks like the strongest choice when we consider both accuracy and training performance, while FunkSVD is the best choice if the main focus is GPU training speed. The Simple reasons are Listed Below why do gpu outperform in some cases but not in others:

## 1. FunkSVD

- It does mainly simple matrix/vector calculations.
- Many calculations are independent of each other.
- So the GPU can do many calculations at the same time.
- Therefore, FunkSVD is very suitable for GPU.
- Main overhead is moving data to the GPU and starting GPU operations.
- For large data, this overhead becomes small compared to the computation.
- Conclusion: FunkSVD can get a large GPU speedup.

## 2. LLORMA

- LLORMA has many calculations that can run in parallel.
- For example:
    - Distance calculation
    - Kernel calculation
    - Local model predictions
- So GPUs can help a lot in these parts.
- But LLORMA creates many local models.
- It also has loops and indexed updates.
- These parts are not as easy to run in parallel.
- Therefore, some GPU power may remain unused.
- Conclusion: LLORMA benefits from GPU, but it has more overhead than FunkSVD.

## 3. GCMC

- GCMC also has many calculations that can run in parallel.
- For example:
    - Embedding calculations
    - Linear layers
    - Decoder calculations
- So the GPU can speed up these parts.
- But GCMC has graph message passing.
- Many messages may need to update the same node.
- This creates extra work for the GPU to manage these updates.
- index_add_() is one example of this.
- Graph data also has irregular memory access.
- Therefore, GCMC cannot always use the GPU as efficiently as simple matrix operations.
- Conclusion: GCMC benefits from GPU, but graph operations add extra overhead.

**– B.Rajasekhar Reddy**
**CS23BT065**